開發者 Buffet

# 一起來快樂惡補
# 基礎 JS、TS、React 吧

饅頭 / Mant0u

本次分享沒有介紹任何 AI 的部分





# 王晟譯

饅頭 / Mant0u

#UI 設計 #饅頭黑體 #GACHAGO

@Mant0u.net

@mant0u.studio

@Mant0uStudio

@mant0u0

@mant0u0

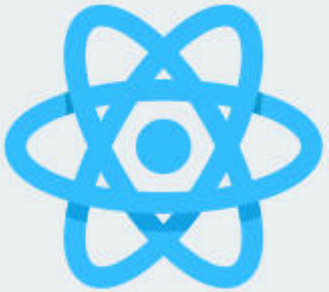
@mant0u




JavaScript ES6+




TypeScript



React



# 01

# JavaScript ES6+

01

1-1

# 變數

## Variable

# JavaScript 變數宣告

賦值

宣告關鍵字 變數名稱 = 值;

var / let / const userName = "MantOu";


函式作用域
區塊作用域
var
let
const
{
}
可修改
可修改
不能改

# 變數的作用域

## var

具備「**函式作用域 (Function Scope)**」

如果在 if 或 for 的大括號 { } 內部宣告 var

變數會**穿透大括號，跑到外面污染全域環境**

```
if (true) {
  var globalMessage = "Hello World";
}

// "Hello World" (穿透了 if 區塊，造成全域污染)
console.log(globalMessage);
```

因為會污染全域，**現代開發已完全棄用**

## let / const

具備「**區塊作用域 (Block Scope)**」

變數被嚴格限制在大括號 { } 內部

變數**無法穿透大括號，離開大括號後該變數便無法存取**

```
if (true) {
  let modalStatus = "open";
  const primaryColor = "#0066FF";
}
// ✕ Uncaught ReferenceError: modalStatus is not defined
console.log(modalStatus);
// ✕ Uncaught ReferenceError: primaryColor is not defined
console.log(primaryColor);
```

為了避免污染變數，**現代開發預設與建議使用**

# let 與 const

## let

可變變數（**可重新賦值**）
當變數的值在程式執行過程中**需要被更新**
（如：計數器、狀態變換）時使用。

```
let count = 2025;
count = 2026;
//  合法：可以重新賦值


let score;
//  合法：可以先宣告不給初始值
score = 100;
```

2025 ▸ 2026

## const

常數宣告（**不可重新賦值**）
變數建立後**不能再指向新的記憶體位址**
宣告時必須立即給值。




```
const number = 20;
number = 50;
// ✕ TypeError: Assignment to constant variable.

const API_KEY;
// ✕ SyntaxError: Missing initializer in const declaration

// 補充：物件內容可修改（指向記憶體位址不變）
const user = { name: "Alex" };
user.name = "Bob";
//  合法！屬性可修改
```



# const 的唯讀特性

## 原始型別

對於**字串 (string)**、**數值 (number)**、**布林值 (boolean)**、**null**、**undefined 等純值**，記憶體直接存取數值本身，因此不允許重新賦值。

```
const maxLimit = 100;
maxLimit = 200;
// ✕ TypeError: Assignment to constant variable.
```

const 鎖定的**物件**記憶體位置，像是生活中的**房子地址**一樣

雖然**門牌地址不能換**，但是房子的**內部裝潢、家具是可以更換**的！



## 複合型別

對於**物件 (Object)**、**陣列 (Array)**，**const 鎖定的是物件在堆疊記憶體中的記憶體位址**。修改物件或陣列內部的成員屬性，並未改變其位址，因此完全合法。

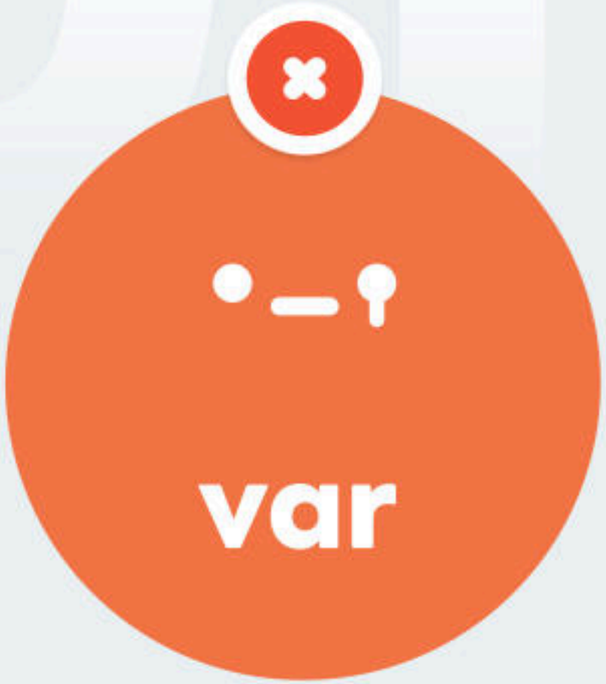
```
const userProfile = {
  name: "Sarah",
  age: 28
};

// ✓ 修改內部屬性：記憶體位址沒變，完全合法
userProfile.age = 29;
console.log(userProfile.age); // 29

// ✕ 重新指派整個新物件：更改了記憶體位址，被 const 攔截報錯
// ✕ TypeError: Assignment to constant variable.
userProfile = { name: "Sarah", age: 29 };
```

後續「1-5 陣列處理方法」會再提到

01




會污染




可變更




唯讀

01

1-2

# 箭頭函式

## Arrow Functions

# 箭頭函式宣告

利用宣告「**變數**」的方式來儲存「**函式**」，所以寫法與一般宣告變數類似。
只是這個變數它**儲存的值不是字串或數字，而是一個函式**。

一般函式宣告

```
function 函式名稱 ( 輸入 ) { 輸出 }
```

**箭頭函式宣告**

```
const 函式名稱 = ( 輸入 ) => { 輸出 };
```

變數宣告

```
const 變數名稱 = 值 ;
```

賦值

**1** 一般函式

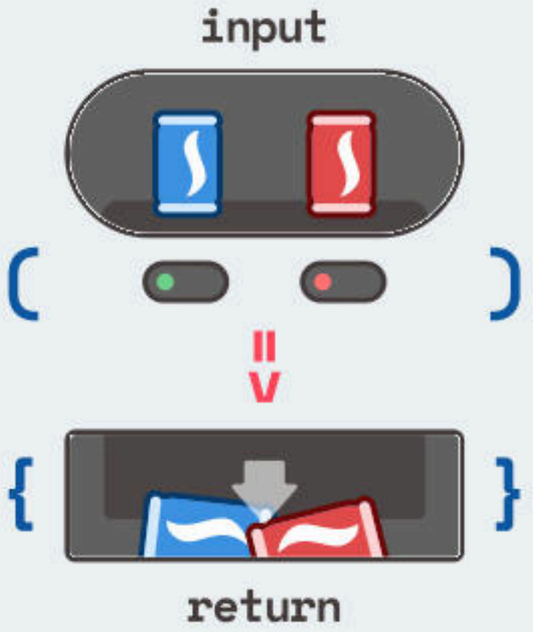
```
function add(a, b){
  return a + b;
}
```

**2** **標準箭頭函式**：設定變數為一個函式，函式的寫法寫作 ( ) => { }

```
const add = (a, b) => {
  return a + b;
};
```

**3** **隱式回傳**：當函式內部只有單一表達式且要回傳時，可省去 { } 與 return 關鍵字

```
const add = (a, b) => a + b;
```




使用隱式回傳時，當回傳為「**物件**」時，避免大括號 { } 被誤認是「**函式區塊**」，所以需要使用 ( ) 將物件包起來。

```
const makeUser = (name) => ({ id: 1, name: name });
```

# this 綁定機制差異

this 代表「**當前正在執行這段程式碼的物件主體**」，但一般函式跟箭頭函式在使用 this 時完全不同～
**一般函式**的 this 非常浮動，它是「**動態決定（看當下是誰呼叫它）**」，在非同步或 Callback 裡，this 就**容易跑位！**
**箭頭函式**的 this 就直覺很多，是「**靜態綁定（看當初寫在哪裡）**」，**指向完全固定，寫起來就不怕它亂跑**。

## 範例 1　基本呼叫（物件方法）

```
const user = {
  name: 'Alice',

  regularFunc: function() {
    console.log(this.name);
  },

  arrowFunc: () => {
    console.log(this.name);
  }
};

user.regularFunc(); // 輸出: "Alice"
user.arrowFunc();   // 輸出: undefined (或空值)
```

**一般函式：** this 是由 user 呼叫，所以 this 指向 user。

```
user.regularFunc();  ▶ 輸出: "Alice"
```

**箭頭函式：** this 是寫在最外層全域（物件 { } 不會產生作用域），所以 this 指向 Window（或 undefined）。

```
user.arrowFunc();  ▶ 輸出: undefined (或空值)
```

**使用函式：** 宣告函式後，就可以直接使用了！

# this 綁定機制差異

## 範例 2　非同步 / Callback 陷阱

在 setTimeout 或陣列方法中使用 callback

```
const timer = {
  seconds: 0,

  start: function() {
    // 這裡的 this 指向 timer 物件
    setTimeout(function() {
      console.log(this.seconds);
    }, 1000);

    setTimeout(() => {
      console.log(this.seconds);
    }, 1000);
  }
};

timer.start();
```

**一般函式問題：**Callback 函式傳入 setTimeout 後是被獨立呼叫（需 1000 毫秒後才會獨立呼叫），當要呼叫時已經遺失了呼叫者，導致 this 指向全域 Window。

`console.log(this.seconds);` ▶ 輸出：undefined（因為 Window.seconds 不存在）

**箭頭函式解法：** 箭頭函式沒有自己的 this，this 又寫在 start() 函式內部，直接繼承外層 start() 函式的 this（即 timer 物件）。

`console.log(this.seconds);` ▶ 輸出：0

**同步**：正常程式執行流程，照順序一行行執行，沒做完就卡住等待。
例如：點擊按鈕，畫面馬上變 1。

**非同步**：程式不會卡在原地等待，發出請求後就繼續往下執行。
例如：點擊按鈕，設定 1 秒後才變 1。（setTimeout 設定 1000ms 的計時器）

**Callback**：回呼函式，作為參數傳給別人的函式，交代對方「處理完後，記得回頭執行我」。
例如：倒數 1 秒後要執行的動作。（變 1 這個動作）

# this 綁定機制差異

## 範例 3　DOM 事件監聽（addEventListener）

使用 JS 抓 HTML 元素綁定事件時，一般函式與箭頭函式獲取的 this 截然不同：

```html
<button id="btn">點我</button>
```
HTML

```js
const btn = document.getElementById('btn');

btn.addEventListener('click', function() {
  console.log(this);
});

btn.addEventListener('click', () => {
  console.log(this);
});
```
JS

**一般函式：**由綁定事件的 btn 元素呼叫，所以 this 指向該 DOM 元素。

`console.log(this);` ▶ 輸出：`<button id="btn">點我</button>`

**箭頭函式：** this 寫在最外層 Script 全域，所以直接繼承 Window。

`console.log(this);` ▶ 輸出：`Window`

**由於 this 的機制不同，可以依照使用需求，選用不同的函式型態：**

**一般函式 ▶ 動態決定（看當下是誰呼叫它）**

**箭頭函式 ▶ 靜態綁定（看當初寫在哪裡）**

01

1-3

# 解構

## Destructuring

01

# 解構

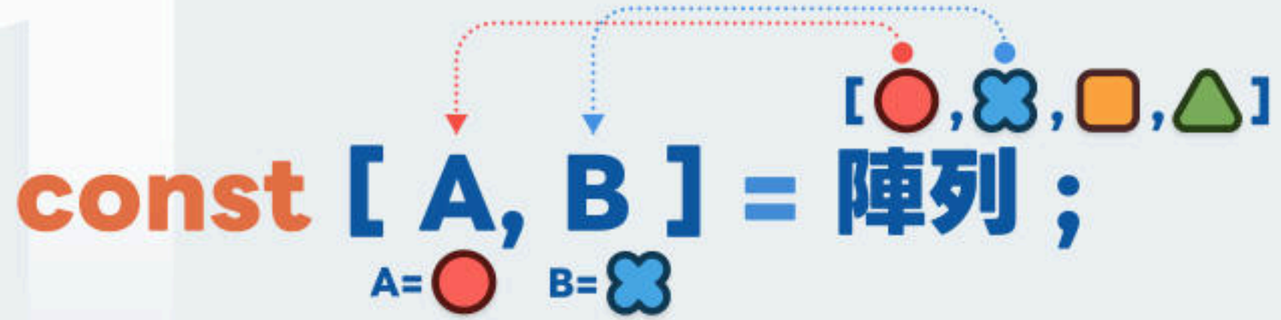
可以把**物件**或**陣列**中的資料

解開擷取成為**獨立變數**

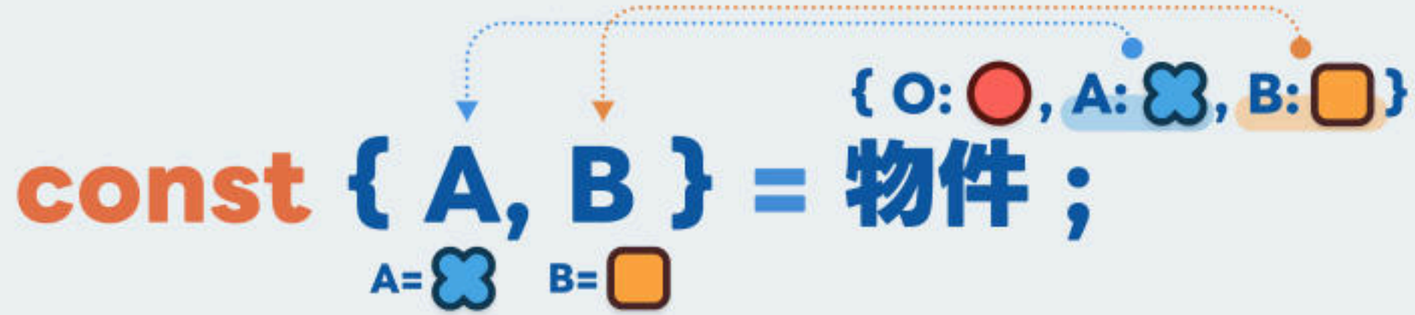




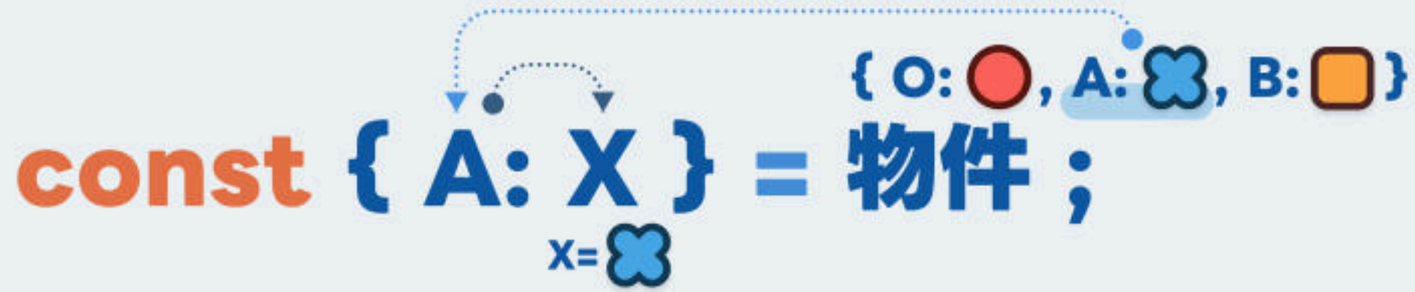
陣列解構




物件解構




物件解構重命名



# 物件解構

物件名稱 = { 物件內容 }

```
const user = {
  id: "U-101",
  name: "Alex",
  role: "admin",
  isVIP: true
};
```

Key : Value

## 1 基本解構

把想要抓出來用的 Key 寫在 { } 中，再設定要從哪一個物件抓資料。

```
const { name, role } = user;
console.log(name, role); // "Alex", "admin"
```

## 2 重新命名（Alias）

如果不想使用物件裡面的 Key 名稱，或是 Key 名稱與其他變數名稱衝突時，

可以使用「:」來重新命名。（key: newName）

```
const { id: userId } = user;
console.log(userId); // "U-101"
```

## 3 設定預設值（Default Value）

使用「=」可以設定「預設值」，當抓的 Key 沒有值（undefined）的時候，

就會自動套用預設值。

```
const { status = "active" } = user;
console.log(status); // "active"
```

# 陣列解構

**陣列名稱 = [ 陣列內容 ]**

```
const scores = [
  95,
  88,
  72,
  60
];
```

## 1 基本解構（按順序賦值）

把想要抓出來用的值，依據位置寫在 [ ] 中，再設定要從哪一個陣列抓資料。

主要是根據「索引位置（第幾個）」，依照順序將陣列裡的值，賦予對應位置的變數。

```
const [firstScore, secondScore] = scores;
console.log(firstScore, secondScore); // 95, 88
```

## 2 跳過特定位置（忽略元素）

使用「, 留空」，可以跳過不需要的陣列項目，直接取得目標位置的值。

```
const [, , thirdScore] = scores; // 跳過前兩個元素，將第三個元素命名為「thirdScore」
console.log(thirdScore); // 72
```

陣列取值得寫法跟物件的寫法差不多，只是因為物件有 Key 可以使用

但陣列沒有 Key，所以只能靠位置。

在 React 元件中，傳入的參數通常是一個**物件（props）**，
可以直接在函式輸入參數的地方，用解構的寫法，讓程式碼比較容易閱讀。

```
const UserCardTraditional = (props) => {
  return <div>{props.name} - {props.role}</div>;
};
```

**未解構寫法**：程式碼需一直重複寫一堆 `props.xxx`

```
const UserCardModern = ({ name, role = "一般會員" }) => {
  return <div>{name} - {role}</div>;
};
```

**解構寫法**：直接把需要的東西抓出來用，程式碼比較簡潔清楚，並帶入預設值

01

1-4

# 展開與其餘運算子

## Spread & Rest Operators

# 展開運算子

像**解壓縮**一樣，把陣列或物件裡面的元素拆開、展開成個別獨立的值。
常用於**複製或合併**陣列/物件，藉由在新陣列/新物件內部展開舊資料，**可產生全新的記憶體位址，確保不破壞原始資料**。

## 沒寫展開運算子

會直接把整坨陣列塞進去，不會解開～

```
const tags = ["JavaScript", "React"];
const newTags = [tags, "Next.js"];

console.log(newTags);
// [["JavaScript", "React"], "Next.js"]
```

## 使用展開運算子

會將原本陣列裡面的元素展開，可以直接放到新的陣列中使用！

```
const tags = ["JavaScript", "React"];
const newTags = [...tags, "Next.js"];

console.log(newTags);
// ["JavaScript", "React", "Next.js"]
```

## 用於物件上，也能覆蓋同名的屬性

將想覆蓋的屬性，寫在後方，可以覆蓋前方的同名屬性

```
const defaultConfig = {
  theme: "dark",
  fontSize: 16,
  showSidebar: true
};

// 複製設定，並僅將 theme 覆蓋為 light
const userConfig = {
  ...defaultConfig,
  theme: "light" // 後方的屬性會覆蓋前方的同名屬性
};

console.log(userConfig.theme);
// "light"
```

# 其餘運算子

解構取值時，選了要使用的參數或屬性後，
可**將剩餘的屬性或元素打包**為獨立的物件或陣列。

## 其餘運算子

用於「**收集/打包**」，打包剩餘的屬性或參數。

放在「宣告變數（等號左邊 / 接收資料）」

```
const order = {
  orderId: "ORD-999",
  item: "MacBook Pro",
  amount: 60000,
  shippingAddress: "高雄市..."
};
// 抽離 orderId 與 item，將剩下的屬性打包進 details 物件中
const { orderId, item, ...details } = order;

console.log(orderId);
// "ORD-999"

console.log(details);
// { amount: 60000, shippingAddress: "高雄市..." }
```

## 展開運算子

用於「**展開/合併**」，將物件或陣列展開拆解。

放在「實際呼叫/傳入（等號右邊 / 提供資料）」

```
const order = {
  orderId: "ORD-999",
  item: "MacBook Pro",
  amount: 60000,
  shippingAddress: "高雄市..."
};
// 複製並覆寫/新增屬性
const updatedOrder = {
  ...order,
  status: "shipped", // 新增屬性
  amount: 58000      // 覆寫既有屬性（折扣價）
};

console.log(updatedOrder);
// { orderId: "ORD-999", item: "MacBook Pro",
amount: 58000, status: "shipped",
shippingAddress: "高雄市..." }
```

01

1-5

# 陣列處理方法

## Array Methods

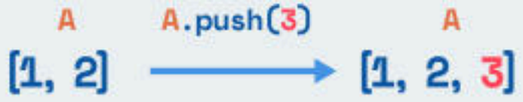
# 操控陣列的方法

## 1 修改原陣列

### .push()

**功能**：將一個或多個元素，**新增**推至陣列的尾端。

**回傳值**：新增元素後的陣列新長度。




### .splice()

**功能**：透過**刪除**、**替換**現有元素或原地新增新元素，來改變陣列內容。

**回傳值**：包含被刪除元素的新陣列。







## 2 產生新陣列 / 累加結果

### .map()

**功能**：將陣列中的每一個元素傳入指定函式處理後，回傳一個結構相同、但內容經**加工**的新陣列。



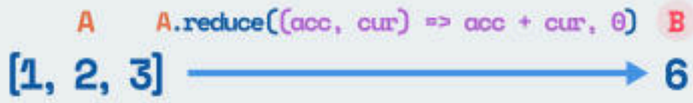

### .filter()

**功能**：**篩選出符合條件**的元素，組成全新的陣列。




### .reduce()

**功能**：將陣列中的每個元素傳入「**累加器**」函式進行計算，最終簡化為單一數值或單一目標物件。



# React 的畫面更新

是根據比對「**資料記憶體位址**」是否改變

直接修改物件、陣列裡面的資料，**是不會觸發畫面重新更新**

**.push()** **.splice()**

不使用直接修改原陣列的方法來更新

**.map()** **.filter()** **.reduce()**

改用產生新陣列或物件的方式來更新

可以回去看「1-1 變數 - const 的唯讀特性」的部分

[🥚, 🌾, 🌽, 🥔].map( 煮 ) => [🍳, 🍚, 🍿, 🍟]

[🍳, 🍚, 🍿, 🍟].filter( 炸 ) => [🍟]

[🍳, 🍚, 🍿, 🍟].reduce( 吃 ) => 💩



# .map( )

**一對一轉換**，把陣列裡的每個元素**拿出來，經過加工處理後，替換成新的值**。
回傳結果為一個與原本長度完全相同的新陣列。

```
const products = [
  { id: 1, name: "鍵盤", price: 2000 },
  { id: 2, name: "滑鼠", price: 800 }
];
```

**1 原始資料陣列**

包含多筆商品物件的初始陣列

```
// 將每個商品打 9 折，回傳新物件陣列
const discountedProducts = products.map((item) => {
  return {
    ...item,
    price: item.price * 0.9
  };
});
```

**2 產生新陣列，並對每個元素加工**

對 products 執行 .map()，並用 item
代表裡面的每一個商品

**3 展開 item 裡面的屬性**

...item 展開屬性 id、name、price

**4 修改 price 覆蓋 ...item 展開的屬性**

item.price*0.9 後，覆蓋前面 price 屬性

```
console.log(discountedProducts);
[
  { id: 1, name: "鍵盤", price: 1800 },
  { id: 2, name: "滑鼠", price: 720 }
];
```

**印出新陣列 discountedProducts 結果**

# .filter( )

**過濾**，設定一個條件，檢查陣列裡的每個元素，**只保留符合條件（回傳 true）的元素**。
回傳結果為一個長度小於等於原本的新陣列。

```
1 const todos = [
2   { id: 1, text: "買牛奶", done: true },
3   { id: 2, text: "寫程式", done: false },
4   { id: 3, text: "運動", done: false }
5 ];
```

**1 原始資料陣列**

包含多筆待辦事項的初始陣列

```
1 // 實作「刪除已完成項目」：只保留 done 為 false 的代辦事項
2 const activeTodos = todos.filter((todo) => {
3   return !todo.done;
  });
```

**2 產生新陣列，並過濾項目**

對 todos 執行 .filter()，並用 todo 代表每一個項目
!todo.done 表示回傳 done 為 false 的未完成項目

```
1 console.log(activeTodos);
  [
    { id: 2, text: "寫程式", done: false },
    { id: 3, text: "運動", done: false }
  ];
```

**印出新陣列 activeTodos 結果**

# .reduce( )

**累加與凝聚**，把陣列裡的每個元素依序「**疊加**」或「**匯總**」起來，最後歸納成一個值。
回傳結果為單一值（可以是數字、字串，甚至是另一個自訂結構的物件）。

```
1 const prices = [100, 250, 400];
2
3 // 計算購物車總金額 (accumulator 初始值為 0)
4 const total = prices.reduce((accumulator, currentPrice) => {
5   return accumulator + currentPrice;
6 }, 0);
7
8 console.log(total); // 750
```

**1 原始資料陣列**

包含三筆商品價格數值的陣列

**2 建立一個 total 變數，並開始累加**

accumulator 為累加器，用來暫存每次計算後的累積結果

currentPrice 為目前處理的單筆價格數值

accumulator + currentPrice 為每輪累加的邏輯

0 為累加器的初始值 (initialValue)

**印出結果**

凝聚成一個數值 100 + 250 + 400 = 750

01

1-6

# 非同步處理

## Asynchronous JS & Async/Await

01

# 同步處理

正常程式的執行流程

程式照順序一行行執行，沒做完就卡住等待

# 非同步處理

需要**等待時間的流程**

程式不會卡在原地等待，發出請求後就繼續往下執行

**例如**：向伺服器請求資料

# 非同步處理的三種狀態

由於非同步操作（如網路請求）無法立刻拿到結果

JavaScript 使用 **Promise 這個物件**，來追蹤這個「**等待過程**」，其中必定會有以下 3 種狀態：



Pending

**進行中**

請求已送出

伺服器還在處理中



Fulfilled

**已成功**

成功拿到資料

回傳 Response



Rejected

**已失敗**

網路中斷或 API 報錯

# async / await

使用 async/await 撰寫**非同步資料抓取邏輯**，並利用 try/catch 處理錯誤
當程式碼執行到 await 時，該非同步請求（Promise）即進入 Pending 階段

**1 發送請求**
await 會暫停等待回應傳回

**2 檢查 HTTP 狀態碼**
錯誤會顯示狀態碼

**3 解析 JSON 資料**
await 暫停等待解析完畢

**4 錯誤攔截區塊**
當 try 有錯誤時，程式會直接進 catch (error)

**5 執行收尾**
不管成功還是失敗都會執行

**執行函式**

```
1  const fetchUserData = async (userId) => {
2    try {
3      const response = await fetch(`https://jsonplaceholder.typicode.com/users/${userId}`);
4
5      if (!response.ok) {
6        throw new Error(`HTTP 錯誤！狀態碼: ${response.status}`);
7      }
8
9      const data = await response.json();
10     return { success: true, data };
11
12   } catch (error) {
13     console.error("抓取資料失敗:", error.message);
14     return { success: false, error: error.message };
15
16   } finally {
17     console.log("請求流程結束。");
18   }
19 };
20
21 fetchUserData(1);
22
```

進行中 / Pending

已成功 / Fulfilled

已失敗 / Rejected

01

1-7

# ES Modules 模組化

## Import & Export

# 導出與引入

使用 **export** 將「變數、函式、物件或元件（Component）」從檔案中導出
讓其他檔案用 **import** 引入使用。

## 具名導出
## Named Exports

**數量**：一個檔案可**有多個具名導出**
**語法**：引入時必須使用大括號 { }
**命名**：引入名稱必須與導出名稱**完全一致**

### 導出端：utils/math.js

```
export const add = (a, b) => a + b;
export const multiply = (a, b) => a * b;
```

### 引入端：app.js

```
import { add, multiply } from "./utils/math";

console.log(add(10, 5)); // 15
```

## 預設導出
## Default Export

**數量**：一個檔案**只能有一個預設導出**
**語法**：引入時不需要使用大括號 { }
**命名**：引入時可以**自由自訂**變數名稱

在 Next.js 中，頁面 (page.js) 與全站排版 (layout.js) 必須使用**預設導出**。

### 導出端：components/Button.js

```
const Button = ({ label }) => {
  return `<button>${label}</button>`;
};

export default Button;
```

### 引入端：app/page.js

```
// 引入時可以自由自訂名稱（例如將 Button 命名為 CustomButton）
import CustomButton from "../components/Button";
```

# 02

# TypeScript

02




# TypeScript

TypeScript 是 JavaScript 的靜態型別超集。它可以在程式碼執行前的編譯階段進行型別檢查，提早發現邏輯與型別錯誤。**透過明確標註型別，有效減少開發 Bug** 並提升程式碼可維護性。

02

變數 = 值

▼

變數：型別 = 值

02

2-1

# 基礎 與 原始型別

## Primitives & Special Types

# 常用原始型別

## 1 字串 String

表示**純文字**，使用單引號 ' '、雙引號 " " 、反引號 ` ` 包覆。
例如：按鈕標籤、標題等。

```
let username: string = "Alex";
```




## 2 數值 Number

表示**所有的數字**，包含整數、浮點數。
例如：寬度、間距、計數等。

```
let itemPrice: number = 2990;
```




## 3 布林值 Boolean

表示**真假狀態**，只有 true 與 false 兩種。
例如：按鈕選項開關狀態。

```
let isModalOpen: boolean = false;
```



## 4 空值 Null

表示「**明確設定為空**」的值。

```
let selectedUser: null = null;
```



**null**：明確的空值，宣告了變數且手動清空的值。
**undefined**：未定義的值，宣告了變數但完全沒理它、沒給值。

# 特殊型別

## 1 any

代表任何值，但會**關閉 TypeScript 型別檢查機制**，所以建議不使用。

```
let badData: any = "Hello";
badData = 123;
// 從字串改成數值也不會報錯，但用在其他地方時，用不好就很容易出錯
```

any 盡量不使用，因為在不確定型別的情況下使用變數，容易出錯

```
let badData: any = "Hello";
badData.toUppercase(); // 應該是 toUpperCase (C 大寫)
```

當寫錯方法名時，TS 完全不警告，執行時直接壞掉！

```
let badData: string | number = "Hello";
badData = 123;
```

定義完整的型別，才不容易出錯

## 2 unknown

代表任何值，但與 any 不同，**在未進行型別檢測（收窄）之前，不允許執行任何操作**，最安全的通用容器。

```
let rawApiResponse: unknown = "https://api.example.com";
console.log(rawApiResponse.length);
// Error: Object is of type 'unknown'.
```

錯誤寫法：unknown 直接使用，TS 會直接出錯攔截

```
let rawApiResponse: unknown = "https://api.example.com";

if (typeof rawApiResponse === "string") {
  console.log(rawApiResponse.length);
  // 安全！TypeScript 確定此時它是字串
}
```

正確寫法：透過型別收窄，安全檢驗後使用

02

2-2

# 複合型別 與 陣列

## Composite Types & Arrays

# 陣列的型別

## 放同型別元素

陣列包含複數個相同型別的元素，長度可自由增減。

**1** **T[ ]：後綴方括號寫法**

```
const productNames: string[] = ["鍵盤", "滑鼠", "螢幕"];
const priceList: number[] = [800, 1200, 4500];
const mixList: (string | number)[] = ["鍵盤", 800];
```

**2** **Array<T>：泛型陣列寫法**

```
const tagList: Array<string> = ["鍵盤", "滑鼠", "螢幕"];
```

## 異質性容器（元組）

長度固定、特定位置對應特定型別的結構。

**型別錯誤：**型別順序不對、長度超標，都會直接報錯

```
const badUser: [string, number, boolean] = [
  25, "Sarah", true
];
```

```
const badUser: [string, number, boolean] = [
  "Sarah", 25, true, true
];
```

**型別正確：**型別順序、長度正確，符合定義

```
const goodUser: [string, number, boolean] = [
  "Sarah", 25, true
];
```

# 物件的型別 與 必填選填

物件的型別與元組容器類似，**缺少屬性、型別不符，都會直接報錯。**

```ts
const submitButton: {
  label: string;
  width: number;
  isDisabled: boolean
} = {
  label: "確認送出",
  width: 120,
  isDisabled: false
};
```

**必填屬性不可少：**當物件類型都有定義時，設定時就不能少。

```ts
const submitButton: {
  label: string;
  width: number;
  isDisabled: boolean
} = {
  label: "確認送出",
  // width: 120, // 缺少時會錯誤
  isDisabled: false
};
```

**選填屬性：**加上 ? 代表這個屬性不一定要傳。

```ts
const submitButton: {
  label: string;
  width?: number; // 選填
  isDisabled: boolean
} = {
  label: "確認送出",
  // width: 120,
  isDisabled: false
};
```

**但是在定義複雜物件時，這樣寫會變得極難閱讀與維護**

```ts
const userCard: {
  id: number;
  info: {
    name: string;
    email: string
  };
  style: {
    theme: string;
    isBordered: boolean
  };
} = {
  id: 1,
  info: {
    name: "Sarah",
    email: "sarah@example.com"
  },
  style: {
    theme: "dark",
    isBordered: true
  }
};
```



02

# 物件變數 : { 物件型別 } = 物件內容

獨立分開寫

interface / type

02

2-3

# 介面 vs 型別別名

## Interface vs Type Alias

# 從「行內寫法」到「獨立定義」

**1** **行內寫法**：冗長且分散難以管理，且無法重複使用。

```ts
const userCard: {
  id: number;
  info: {
    name: string;
    email: string
  };
  style: {
    theme: string;
    isBordered: boolean
  };
} = {
  id: 1,
  info: {
    name: "Sarah",
    email: "sarah@example.com"
  },
  style: {
    theme: "dark",
    isBordered: true
  }
};
```

**2** **獨立定義**：型別模組化管理，再讓變數直接套用！

```ts
interface UserCardProps {
  id: number;
  info: {
    name: string;
    email: string
  };
  style: {
    theme: string;
    isBordered: boolean
  };
}
const userCard: UserCardProps = {
  id: 1,
  info: {
    name: "Sarah",
    email: "sarah@example.com"
  },
  style: {
    theme: "dark",
    isBordered: true
  }
};
```

```ts
// 把子結構獨立出來
interface UserInfo {
  name: string;
  email: string;
}

interface UserStyle {
  theme: string;
  isBordered: boolean;
}

// 在主介面中組合它們
interface UserCardProps {
  id: number;
  info: UserInfo;
  style: UserStyle;
}
```



使用 interface 可以定義型別介面，注意使用**開頭大寫駝峰式命名**（例：UserStyle）

# Interface 介面

Interface 用來定義**物件的型別**，**不寫等號 =** ，直接使用**物件大括號 { }** 來設定。
若想讓其他介面**繼承現有規格**，可以使用 **extends** 語法來**複製並擴充欄位**。

## 1 Interface 設定方式

```ts
// 定義基礎按鈕介面
interface ButtonProps {
  readonly id: string;   // 唯讀：元件 ID 不可修改
  label: string;         // 必填：按鈕文字
  width: number;         // 必填：寬度
  isDisabled?: boolean; // 選填：停用狀態
}

const submitBtn: ButtonProps = {
  id: "btn-01",
  label: "確認送出",
  width: 120
  // isDisabled 是選填，不傳也不會報錯
};
```

## 2 extends 繼承：用於複製與擴充型別

```ts
// 繼承 ButtonProps 並追加 Icon 欄位
interface IconButtonProps extends ButtonProps {
  iconName: string; // 新增 Icon 屬性
}

const downloadBtn: IconButtonProps = {
  id: "btn-02",
  label: "下載檔案",
  width: 140,
  iconName: "download-icon"
  // 擁有了 ButtonProps 的所有屬性 + 自己的 iconName
};
```

# Type Alias 型別別名

Type 用來**給任何型別取別名，需要寫等號 =**（有點像是定義變數）。
除了物件外，還能定義**聯集型別**（像是指定選項 'sm' | 'lg'），並使用 **& 符號來組合擴充型別**。

## 1 Type Alias 設定方式、「|」聯集型別

```ts
// 使用原始型別定義
type BoxWidth = number;
// 使用聯集型別定義（指定限定選項）
type ButtonSize = 'sm' | 'md' | 'lg';

// 實際套用到變數
const myWidth: BoxWidth = 200;
const mySize: ButtonSize = 'md';

// 填入非限定選項會直接報錯：
// const badSize: ButtonSize = 'xl'; // Error!
```

**使用聯集型別，可以限制變數的選項**
**當輸入選項外的資訊，就會顯示錯誤！**

## 2 「&」交集型別：用於組合擴充型別

```ts
type BaseStyle = { width: number };

type AdvancedButton = BaseStyle & {
  size: ButtonSize;
};

// 必須包含 BaseStyle 與擴充屬性
const actionBtn: AdvancedButton = {
  width: 120,
  size: 'lg'
};
```

# Interface 與 Type 的使用

Interface 就是拿來**定義一整個元件的規格**，Type Alias 則是用來**定義個別項目規格**
兩者可以選擇合適的時候使用！

**1** **左區塊（模組化）：** 把 ButtonSize 獨立出來，方便未來其他元件（如 Card、Icon）跨元件共用。

```ts
type ButtonSize = 'sm' | 'md' | 'lg';

interface ButtonProps {
  readonly id: string;
  label: string;
  width: number;
  isDisabled?: boolean;
  size?: ButtonSize;
}
```

**2** **右區塊（行內內聯）：** 如果這個尺寸只有這個按鈕自己會用到，直接寫 'sm' | 'md' | 'lg' 就可以了！

```ts
// type ButtonSize = 'sm' | 'md' | 'lg';

interface ButtonProps {
  readonly id: string;
  label: string;
  width: number;
  isDisabled?: boolean;
  size?: 'sm' | 'md' | 'lg';
}
```

02

2-4

# 聯集型別

## Union Types

# 聯集型別

透過 **|（聯集）符號**串接**字面量型別（Literal Types）**，能指定**多個特定字串**，宣告該型別只能是這些字串中的其中一個。
常用於限制**元件屬性（如按鈕樣式、尺寸）**，能在開發時提供自動補全，並防止拼錯字或輸入無效值。

```typescript
// 限制按鈕樣式只能是這三種字串之一
type ButtonVariant = "primary" | "secondary" | "danger";

interface CustomButtonProps {
  label: string;
  variant: ButtonVariant; // 使用聯集型別
}

const myButton: CustomButtonProps = {
  label: "刪除檔案",
  variant: "danger" // 合法
};

// 拼錯字或傳入未許可的字串，編輯器第一時間噴紅線攔截
const badButton: CustomButtonProps = { label: "儲存", variant: "blue" };
// Error: Type '"blue"' is not assignable to type 'ButtonVariant'.
```

### 定義聯集型別

限制變數只能為 "primary" | "secondary" | "danger" 三個字串之一。

### 正確帶入

傳入合法的 "danger" 字串。

### 錯誤攔截

傳入未定義的 "blue" 時，TypeScript 會在編譯階段第一時間拋出錯誤阻擋。

# 聯集型別的應用

前面介紹了「**聯集型別（|）**」與「**選填屬性（?）**」，但是當組合越來越多時，很容易因設計不當而產生「**無效或矛盾的狀態**」。
只要在聯集的各個物件中，**加入共通的可辨識標籤**（如 status），TypeScript 就能在編譯時先抓出矛盾物件，達成狀態防護。

**範例** 設計一個狀態接收「**成功**」要有**資料**，狀態接收「**錯誤**」要有**錯誤訊息**，的型別設計。

### 容易產生矛盾狀態的設計

status 為 success、error 兩種狀況聯集
data 與 errorMessage 為選填

```ts
type ApiResponse = {
  status: "success" | "error";
  data?: string[];
  errorMessage?: string;
};
```

**型別太寬鬆，容易允許不可能發生的邏輯錯誤存在：**

- status 為 success 時，依然可以誤寫 errorMessage
- status 為 error 時，依然可以誤寫 data

### 使用「可辨識標籤」的設計

將成功與錯誤兩種狀態，**型別分開定義，再將兩個型別聯集**
這樣物件就必須完全符合 FetchSuccess 或 FetchError 其中一種規格

```ts
type FetchSuccess = { status: "success"; data: string[] };
type FetchError   = { status: "error"; errorMessage: string };

// 用 | 串接，從源頭杜絕矛盾狀態
type ApiResponse = FetchSuccess | FetchError;
```

**利用必填型別與聯集的特性，減少選填型別的使用**
可以少寫多餘的 if 防呆判斷，只要檢查 status 即可。

02

2-5

# 泛型

## Generics

```
function 函式 ( 輸入變數 ) {
  函式內容
  使用 輸入變數
}
```

```
interface 型別 < 輸入型別 > {
  型別內容
  型別使用 輸入型別
}
```

### 函式

函式可以根據輸入的變數
**輸出或執行對應的結果**

### 型別

而型別也有類似的寫法，也就是「**泛型**」
根據輸入的型別，**產生對應的型別**



**定義時先不寫死，等到用的時候，再把東西「餵」進去使用**

# 泛型

先宣告型別**輸入 <T>**，再將它對準內部的資料欄位。
在**呼叫的當下傳入真正的型別**，即可動態生成精準的型別規格。

**不使用泛型：**如果不使用泛型，你可能要為每一種資料都寫一個 Interface

```ts
// 用戶 API 的外殼
interface UserResponse {
  code: number;
  data: UserData; // 寫死了 UserData
}

// 商品 API 的外殼（幾乎一模一樣，只改了 data 型別）
interface ProductResponse {
  code: number;
  data: ProductData; // 寫死了 ProductData
}
```

外殼一模一樣，卻要重複寫好多次
程式碼變得很冗長

**使用泛型 <T> 彈性包裝：**將 data 的型別抽出變成參數 <T>

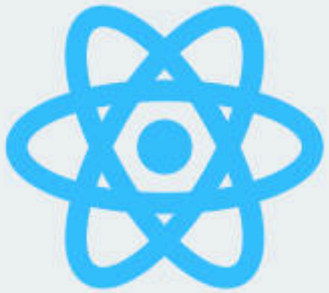
```ts
// 1. 定義一個「通用外殼」，<T> 是一個型別預留位
interface ApiResponse<T> {
  code: number;
  message: string;
  data: T; //這裡的型別由外面帶進來的 T 決定！
}

// 2. 實際套用時，才把真正的型別「餵」進去：
// 傳入 UserData
const userRes: ApiResponse<UserData> = {
  code: 200,
  message: "Success",
  data: { name: "Alex", id: "U-01" }
};

// 傳入 ProductData[] 陣列
const productRes: ApiResponse<ProductData[]> = {
  code: 200,
  message: "Success",
  data: [{ productId: "P-101", price: 500 }]
};
```

# 03

# React



# React

React 是一個專門用來建構使用者介面的 JavaScript 函式庫。與傳統 JavaScript 的「**指令式**」寫法不同，React 採用「**宣告式**」設計，開發者只需專注於**資料狀態與畫面的對應關係**，React 就會自動更新渲染畫面。

03

# 指令式 vs 宣告式

# 指令式 Imperative

原生的 JavaScript 開發方式為「**步驟導向**」，必須一步步手動選取與修改每個 HTML 節點，才能更新 UI。

**1 建立 / 尋找元素**

`document.createElement(...)` 或 `document.getElementById(...)`

**2 監聽使用者事件**

手動綁定事件 `button.addEventListener('click', ...)`

**3 改變變數數值**

在事件觸發時手動修改資料 `count += 1`

觸發點擊事件時，手動累加資料變數

**4 手動更新畫面**

手動將新數值塞回 DOM `button.innerText = ...`

更新 HTML 上的畫面

**缺點：**當畫面變得龐大、資料互相關聯時

只要漏寫其中一個步驟，畫面與資料就會失去同步。

```javascript
const button = document.createElement('button');
button.className = 'btn-primary';
button.innerText = '點擊計數: 0';

let count = 0;
button.addEventListener('click', () => {
  count += 1;
  button.innerText = `點擊計數: ${count}`;
  // 手動尋找並變更實體 DOM 節點
});

document.getElementById('app').appendChild(button);
```

JavaScript

```html
<!-- 執行 JS 之前 -->
<div id="app"></div>

<!-- 執行 JS 之後，會變成這樣 -->
<div id="app">
  <button class="btn-primary">點擊計數: 0</button>
</div>
```

HTML

# 宣告式 Declarative

React 開發方式為「**資料驅動畫面**」，不用管「如何抓取 DOM」或「步驟是什麼」
只要定義好資料與 UI 的關係，畫面就會自動更新。

**1** **定義資料狀態 (State)**

`count`：是當前資料狀態（初始值為 0）。
`setCount`：是更新狀態的專用函式。

**2** **定義畫面 UI**

透過與 HTML 相似的 **JSX 語法**
宣告這個狀態下畫面該長什麼樣子。

開發者**不需要自己手動修改 DOM 節點，只要呼叫 setCount 改變狀態**
當資料改變時，React 會自動更新 HTML 畫面。

```typescript
import React, { useState } from 'react';

export const Counter = () => {
  const [count, setCount] = useState(0);

  return (
    <button
      className="btn-primary"
      onClick={() => setCount(prev => prev + 1)}
    >
      點擊計數: {count}
    </button>
  );
};
```

TypeScript

03

3-1

# JSX 語法

## JavaScript XML

# JSX 語法

React 提供**類似 HTML 標籤的語法**擴充，讓我們能在 JavaScript 中直覺地定義 UI 結構
以下為 JSX 的語法規範：

## 1 單一根元素

JavaScript 函式一次只能回傳一個值。
所以多標籤時，需用**空標籤 <>...</>** 包覆。

```jsx
<>
  <h1>標題</h1>
  <p>內文</p>
</>
```

## 2 標籤自我閉合

JSX 語法極嚴格，所有單一標籤必須加上 / 閉合。

```jsx
<img src="a.jpg" />
<input type="text" />
<br />
```

## 3 駝峰式命名

屬性一律改為駝峰式命名（如 onClick），並避開 JS 保留字（如 class 改為 **className**）。

```jsx
<button className="btn" onClick={handleClick}>
  送出
</button>
```

## 4 行內樣式物件

style 不接收字串，需傳入 JS 物件，CSS 屬性採駝峰命名。

```jsx
<div style={{
  backgroundColor: 'red',
  fontSize: '16px',
  padding: '12px'
}}>
  Hello
</div>
```

# JSX 語法解析

由於**瀏覽器無法直接看懂 JSX**，必須透過轉譯器轉換為標準 JavaScript 物件（**虛擬 DOM**），提供 React 引擎**比對差異**，最後精準渲染至 HTML 畫面（**真實 DOM**）。

### JSX 語法
我們直覺撰寫的 UI 宣告

### 編譯 JavaScript
轉譯為標準 JS 函式
（Babel / SWC 轉譯器）

### 虛擬 DOM 物件
產出描述畫面的 JS 物件
（VDOM）

### 渲染真實 DOM
比對差異並自動更新
HTML 畫面

```
const [count, setCount] = useState(1);
```

```
const element = (
  <p className="count">
    {count}
  </p>
);
```

```
const element = _jsx("p", {
  className: "count",
  children: count,
});
```

```
{
  type: 'p',
  props: {
    className: 'count',
    children: 1
  },
  key: null,
  ref: null
}
```

```
<p class="count">1</p>
```

**將 count 更新成 2**

```
setCount(2);
```

```
const element = _jsx("p", {
  className: "count",
  children: count,
});
```

```
{
  type: 'p',
  props: {
    className: 'count',
    children: 2
  },
  key: null,
  ref: null
}
```

**比對虛擬 DOM 發現差異，更新實體 DOM**

```
<p class="count">2</p>
```




因為**直接操作實體網頁 DOM 的代價非常高**，所以 React 選擇在背景比較新舊的虛擬 DOM，**發現新舊差異點後，才會作最小限度更新**。

# 大括號插槽 { }

在 JSX 結構中，大括號 { } 代表「**切換回 JavaScript 模式**」。
內部可放入任何 **JavaScript 變數或表達式**，將動態資料即時注入到畫面上。

## 1 放入「基礎變數 與 運算式」

在文字或標籤屬性中，直接放入 JS 變數或做簡單運算。

```jsx
export const Welcome = () => {
  const name = "Alice";
  const avatarUrl = "https://example.com/avatar.jpg";
  const btnType = "primary"; // primary | secondary
  const score = 90;

  return (
    <div className="card">
      {/* 1. 屬性綁定變數：圖片網址 */}
      <img src={avatarUrl} alt={name} />

      {/* 2. 內容嵌入變數 */}
      <h1>你好，{name}</h1>

      {/* 3. 內容嵌入運算式 */}
      <p>預估分數：{score + 5}</p>

      {/* 4. 屬性動態拼接 Class */}
      <button className={`btn btn-${btnType}`}>
        查看個人檔案
      </button>
    </div>
  );
};
```




## 2 放入「自訂元件 或 JSX」

JSX 元素也是一個普通的 JS 物件，可以存成變數再放入大括號。

```jsx
// 1. 定義一個子元件（使用大寫開頭）
const StarIcon = () => <span>⭐</span>;

export const Card = () => {
  // 2. 把一段 JSX 存在變數裡
  const badge = <span>[特價中]</span>;

  return (
    <div>
      {/* 方式 1：直接以標籤方式使用自訂元件 */}
      <h2>商品名稱 <StarIcon /></h2>

      {/* 方式 2：使用大括號插槽，渲染存成變數的 JSX */}
      {badge}
    </div>
  );
};
```

# 大括號插槽 { }

在 JSX 結構中，大括號 { } 代表「**切換回 JavaScript 模式**」。
內部可放入任何 **JavaScript 變數或表達式**，將動態資料即時注入到畫面上。

## 3 邏輯與 &&（顯示/隱藏）

條件成立才顯示，不成立則什麼都不畫。
{ 條件 && 成立時顯示的內容 }

```jsx
export const Message = () => {
  const count = 3;

  return (
    <div>
      {/* 1. 單一標籤 */}
      {count > 0 && <span>新訊息：{count} 則</span>}

      {/* 2. 多個標籤：使用空標籤 <>...</> 包覆 */}
      {count > 0 && (
        <>
          <span className="dot">🔴</span>
          <span>新訊息：{count} 則</span>
          <button>前往查看</button>
        </>
      )}
    </div>
  );
};
```

{ && } <v-if>

## 4 三元運算子（二選一）

根據條件決定顯示 A 還是 B。
{ 條件 ? 成立內容 : 不成立內容 }

```jsx
export const LoginStatus = () => {
  const isLoggedIn = true;

  return (
    <div>
      {/* 登入顯示「登出」，未登入顯示「登入」 */}
      {isLoggedIn ? <button>登出</button> : <button>登入</button>}
    </div>
  );
};
```

{ ? : } <v-if> <v-else>

因為 { } 內部只能放「**表達式**」，就是必須具有回傳值的程式碼
所以**不能直接放「語句」，如 if...else、for 迴圈等程式碼**

03

3-2

# 元件 與 屬性傳遞

## Components & Props Architecture

# 元件

是 React 構建畫面的**基本積木！**

例如：按鈕、卡片等區塊，經常會使用元件化的方式維護。

## 1 使用函式，定義介面元件

React 元件本質上就是一個接收輸入參數並回傳 JSX (UI) 的 JavaScript 函式。

## 2 大寫字母開頭

為了與標準 HTML 標籤做區隔，元件名稱必須使用「**大寫字母開頭**」。（例如：UserCard、TodoItem、Item）

```
<ul>
  <li>
    <span>1.</span>
    <span>Mant0u</span>
  </li>
  <li>
    <span>2.</span>
    <span>Bun</span>
  </li>
</ul>
```

```
<ul>
  <Item num={1} name={"Mant0u"} />
  <Item num={2} name={"Bun"} />
</ul>
```

元件化管理介面，**減少重複的程式碼**

```
import React from 'react';

// 1. 定義規格 (Props)：宣告這個積木需要什麼資料
interface ItemProps {
  num: number;
  name: string;
}

// 2. 定義元件：接收參數並回傳 JSX 結構
export const Item = ({ num, name }: ItemProps) => {
  return (
    <li>
      <span>{num}.</span>
      <span>{name}</span>
    </li>
  );
};

// 3. 重用元件：像 HTML 標籤一樣呼叫與傳入資料
export const App = () => {
  return (
    <ul>
      <Item num={1} name="Mant0u" />
      <Item num={2} name="Bun" />
    </ul>
  );
};
```



# Props

把 React **元件看成一個函式**，Props 就是要**傳入元件中給元件使用的參數**。
只要給它不同的資料（Props），它就會在畫面上印出不同的內容。

### 1 單向傳遞

資料只能由**父元件流向子元件**

▶ 資料 msg 只能由 <Parent> 流向 <Child>

### 2 唯讀限制

子元件絕對**不能在內部直接修改**傳進來的 Props

▶ <Child> 裡不能直接寫 text = "..." 覆寫資料

### ! 修改方式

若子元件需要改變資料，必須由父元件傳遞一個「**觸發函式**」給子元件。**由子元件呼叫該函式，通知父元件自行修改資料**。

▶ <Child> 呼叫 onUpdate()，通知 <Parent> 自己去執行 setMsg('已更新')




```tsx
import { useState } from 'react';

// 定義型別
type ChildProps = {
  text: string;
  onUpdate: () => void;
};

// 子元件：接收資料與通知函式
const Child = ({ text, onUpdate }: ChildProps) => {
  // text = "新文字"; (Props 唯讀)
  return <button onClick={onUpdate}>{text}</button>;
};

// 父元件：管理資料並往下傳
const Parent = () => {
  const [msg, setMsg] = useState('點我');

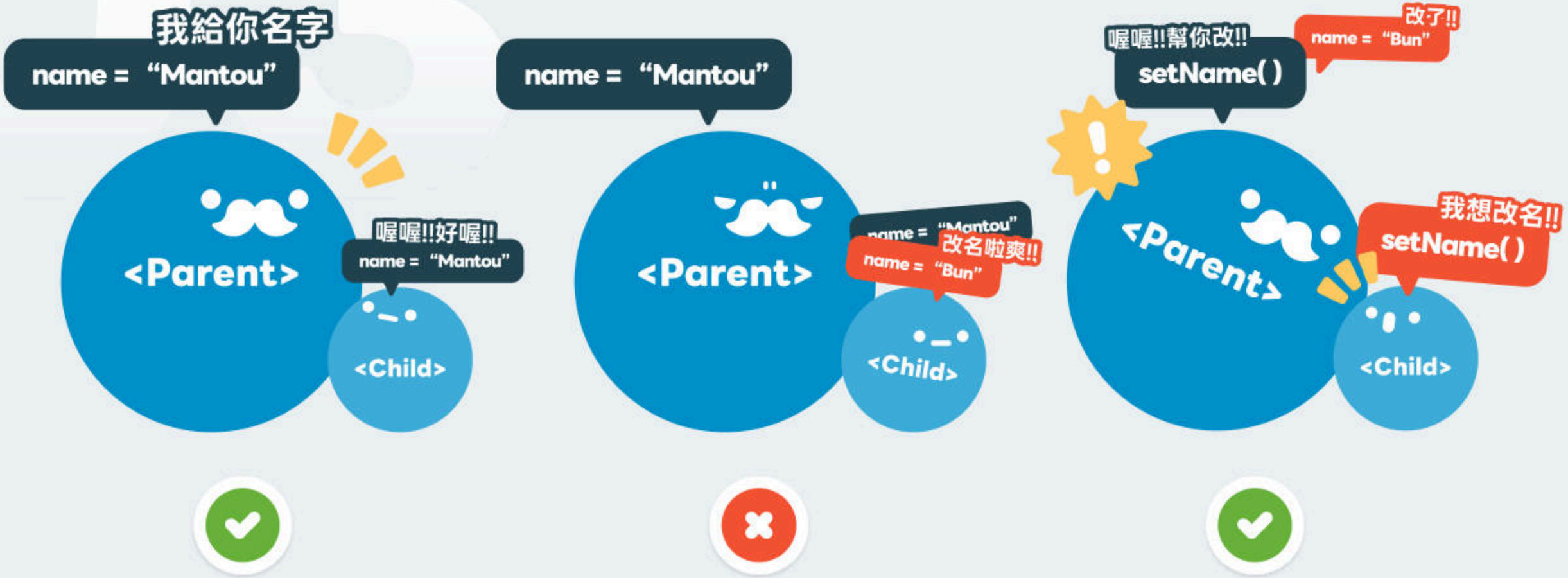
  return (
    <Child
      text={msg}
      onUpdate={() => setMsg('已更新')}
    />
  );
};
```

**Props**
子元件的輸入參數

**Props 唯讀**
禁止修改傳入參數
父元件 ↔ 子元件

**單向傳遞**
資料往下流
父元件 → 子元件

**傳遞函式**
傳入一個給子元件
通知父元件改資料的函式


我給你名字
name = “Mantou”
<Parent>
喔喔!!好喔!!
name = “Mantou”
<Child>
name = “Mantou”
<Parent>
name = “Mantou”
name = “Bun”
改名啦爽!!
<Child>
喔喔!!幫你改!!
setName( )
改了!!
name = “Bun”
<Parent>
我想改名!!
setName( )
<Child>

## 範例 結合 TypeScript 定義 Props 規範

```tsx
import React from 'react';

// 1. 定義可接收的 Props 規範
export interface ButtonProps {
  label: string;
  variant?: 'primary' | 'secondary' | 'danger';
  disabled?: boolean;
  onClick: () => void;
}

// 2. 元件解構帶入參數，並指定預設值
export const CustomButton = ({
  label,
  variant = 'primary',
  disabled = false,
  onClick,
}: ButtonProps) => {
  return (
    <button
      className={`btn btn-${variant}`}
      disabled={disabled}
      onClick={onClick}
    >
      {label}
    </button>
  );
};
```

TypeScript

- 必填屬性
- 選填屬性 (加 ?)
- 函式型別
- 指定預設值

# Children 容器模式

children 是 React **預留的關鍵字**，當你希望一個元件能夠像外殼一樣「**包裹**」其他 JSX 節點時，就可以使用 children。
它的 TypeScript 型別通常是 React.ReactNode。

```tsx
interface ModalProps {
  title: string;
  isOpen: boolean;
  onClose: () => void;
  children?: React.ReactNode; // 接受任何合法的 JSX 節點，選填
}

export const Modal = ({ title, isOpen, onClose, children }: ModalProps) => {
  if (!isOpen) return null;

  return (
    <div className="modal-backdrop">
      <div className="modal-box">
        <h3>{title}</h3>
        <button onClick={onClose}>關閉</button>

        {/* 渲染外面包進來的任何內容 */}
        <div className="modal-body">{children}</div>
      </div>
    </div>
  );
};
```

## 使用 children 表示內元素

元件標籤夾住的內容，會自動傳入為 props.children
概念等同於 Vue 的預設插槽（Default Slot）

children <slot>

children 模式讓元件更靈活、可重用性更高，**可以使元件核心內容留白，讓元件可以塞入任何表單、說明文字、圖片等內容**。（例如：**視窗元件**）

### 使用方法

```tsx
<Modal title="系統公告" isOpen={true} onClose={handleClose}>
  <p>這段文字會作為 children 傳入 Modal 內部</p>
</Modal>
```

03

3-3

# React Hooks

## React Hooks

# Hooks

React 提供的**特殊函式**，讓元件能夠「**鉤入 (Hook into)**」React 的**底層核心機制**
將需要的能力「**勾**」出來使用，例如：狀態管理、外部系統同步或記憶體託管。

## useState

**狀態儲存空間**

跨週期保留資料
變更時觸發畫面重繪

## useEffect

**外部系統同步 / 副作用**

處理定時器、事件訂閱或
外部連線同步

## useRef

**記憶體容器 / DOM**

跨週期保留資料
變更時不觸發畫面重繪

## useMemo

**運算結果（值）**

快取昂貴運算結果
避免重複計算

## useCallback

**函式實體（記憶體位址）**

快取函式參考
避免子元件無謂重繪

# 為什麼需要 Hooks？

從最核心的「**狀態（State）**」談起，在 React 中修改一般的區域變數不會觸發**重新渲染 (Re-render)**，且元件重新渲染時變數也會被重置。因此**我們需要一種機制**，讓資料能跨渲染週期被保留並通知畫面更新。

**範例** 做一個點擊按鈕，就會累加 1 的計數器。

```jsx
export const CounterWrong = () => {
  let count = 0;

  const handleClick = () => {
    count = count + 1;
    console.log('目前 count:', count); //數值有變
  };

  return (
    <div>
      <p>點擊次數：{count}</p> {/* 但畫面不會更新 */}
      <button onClick={handleClick}>點我加 1</button>
    </div>
  );
};
```

**一般區域變數：**

就算是變數有變，但畫面依然不會更新！

```jsx
import { useState } from 'react';

export const CounterCorrect = () => {
  const [count, setCount] = useState(0);

  const handleClick = () => {
    setCount(count + 1);
  };

  return (
    <div>
      <p>點擊次數：{count}</p> {/* 畫面更新成功 */}
      <button onClick={handleClick}>點我加 1</button>
    </div>
  );
};
```

**使用 React Hook：**

透過 **useState** 這個 Hook 更新狀態，並通知 React 重新渲染，這樣畫面才會順利更新！

# useState

React 提供的一個，用來讓元件擁有**保存狀態（State）**的 Hook
並且能在狀態改變時，**觸發畫面重新渲染（Re-render）**。
也能**避免元件經重新渲染後，狀態被初始化掉**！

```jsx
import { useState } from 'react';

const Counter = () => {
  // 解構賦值：[當前狀態值, 更新狀態的函數]
  const [count, setCount] = useState(0);

  const handleClick = () => {
    setCount(count + 1);
  };

  return (
    <button onClick={handleClick}>
      點擊了 {count} 次
    </button>
  );
};

export default Counter;
```

```jsx
const [count, setCount] = useState(0);
```

**傳入參數** **狀態的初始值**

例如：數字 0、字串 " "、陣列 [ ] 或物件 { }

**回傳值** **當前狀態值**

**顯示當下的值**，通常直接寫在介面中。

▶ 當前渲染週期下的狀態值

**回傳值** **更新狀態的函式**

要**觸發狀態更新**時，需要使用這個函式來幫忙更新畫面。

▶ 用來更新狀態並且能觸發重新渲染的更新函式（Setter）。

# useState

使用 useState 時，建議留意以下重點：
包含 useState 的運作特性，以及觸發重新渲染的時機！

## 1 內容不可變性

更新物件、陣列時，不能直接修改原始狀態，必須建立全新的物件、陣列，**React 比較新舊物件，發現有改變時才會重新渲染**！

```
// 錯誤：直接修改物件，React 檢測不到變化
person.name = 'Mant0u';
setPerson(person);

// 正確：展開運算子產生新物件
setPerson({ ...person, name: 'Mant0u' });
```

React 底層使用的是 Object.is() **淺層比較**
只會比較物件陣列最外層，不比較複雜的內層！

## 2 判斷是否重新渲染

**基本型別**：**比對值**，像是 Number, String 等，比對數值是否改變，若新舊值相同則不會觸發重新渲染。

**物件與陣列**：**比對記憶體位址**，直接修改原物件內容，無法被 React 識別，必須使用展開運算子 ( ... ) 等方式建立新物件、陣列來觸發更新。

## 3 快照與函式型更新 ( Functional Update )

因為每一次渲染週期，**State 會有固定的快照資料，導致 State 數值會固定不變**。
如果直接使用舊狀態值，可能會吃到舊的快照資料，導致更新結果錯誤。

```
export const CounterDemo = () => {
  const [count, setCount] = useState(0);

  const handleAddThree = () => {
    setCount(count + 1);
    setCount(count + 1);
    setCount(count + 1);
  };

  return <button onClick={handleAddThree}>Count: {count}</button>;
};
```

```
const handleAddThree = () => {
  setCount(prev => prev + 1);
  setCount(prev => prev + 1);
  setCount(prev => prev + 1);
};
```

**錯誤寫法**
三次傳入的 count 都是當前週期的快照 0，最終結果只會是 1

**正確寫法**
傳入 Callback 函式 (prev ⇒ prev + 1)，React 會確保拿到更新佇列中的最新數值

# useRef

React 提供的一個 Hook，用來讓元件能儲存跨渲染週期的**可變值**，**且改變時「不會」觸發重新渲染**。
同時也是 React 中用來直接存取與操作真實 DOM 元素的主要方式！

```jsx
import { useRef } from 'react';

const TextInputWithFocusButton = () => {
  // 建立 Ref 物件：傳入初始值（如 null、0 等）
  const inputEl = useRef(null);

  const handleClick = () => {
    // 透過 .current 屬性讀取或修改值/DOM
    inputEl.current.focus();
  };

  return (
    <>
      {/* 綁定 ref 到 DOM 節點 */}
      <input ref={inputEl} type="text" />
      <button onClick={handleClick}>聚焦輸入框</button>
    </>
  );
};

export default TextInputWithFocusButton;
```

```
const inputEl = useRef( 初始值 );
```

**傳入參數** **傳入參數初始值**

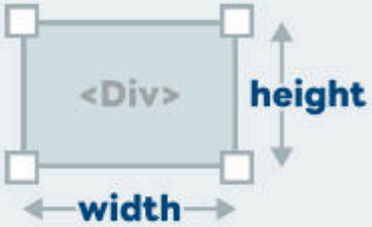
DOM 預設為 null，計數器/Timer ID 預設為 0

**回傳值** **一個可變的 Ref 物件**

傳一個包含 .current 屬性的物件。
內容可以直接修改（例如 ref.current = 1），
**且不會觸發畫面重新渲染**。

# useRef

使用 useRef 時，建議留意以下重點：
包含 useRef 的兩大用途，以及與 useState 的核心差異！




## 1 操作真實 DOM 節點（例如：取得元素尺寸、捲軸位置等）

宣告 const ref = useRef(null) 並**綁定到 JSX 元素** <input ref={ref} />。

渲染後可直接使用瀏覽器原生方法操作該元素，不需要 React 重新渲染。

▶ 常見情境：取得聚焦（Focus）狀態、滾動滾輪（Scroll）、量測寬高、播放影片。




## 2 儲存不需即時呈現在畫面上的變數

與 useState 一樣，元件重新渲染時保存在裡面的值不會遺失。

與 useState 不同的是，修改 ref.current 時**不會觸發畫面重新渲染**（適合當幕後計時、快取或暫存器）。

▶ 常見情境：記錄計時器 ID（timerId）、前一次的狀態、是否首次載入的標籤。




| | 修改值是否觸發重新渲染 | 資料是否跨渲染保留 | 主要用途 |
|---|---|---|---|
| useRef | 不會<br>默默更新 | 會保留在 .current | 操作真實 DOM<br>記錄幕後變數 |
| useState | 會<br>通知 React 重新執行元件 | 會保留在 State 中 | 存放直接影響<br>畫面呈現的資料 |

如果**某個資料的改變需要畫面跟著變**，用 useState！
如果**改了畫面不需要變**（例如記住定時器編號），用 useRef！

# useEffect

React 用來讓元件與「**外部系統**」保持同步的 Hook。
例如：發送 API 請求、設定計時器、手動操作 DOM 或註冊全域事件監聽等。
這些**在 React 渲染機制之外的同步行為**，統稱為「**副作用（Side Effects）**」

```jsx
import { useState, useEffect } from 'react';

const Timer = () => {
  const [seconds, setSeconds] = useState(0);

  useEffect(() => {
    // 1. 副作用邏輯
    const timerId = setInterval(() => {
      setSeconds(prev => prev + 1);
    }, 1000);

    // 2. 清理函式
    return () => {
      clearInterval(timerId);
    };
  }, []); // 3. 依賴陣列

  return <p>經過時間：{seconds} 秒</p>;
};

export default Timer;
```

### 1 副作用邏輯（Setup）

**當元件剛渲染完成後（元件被產生、進場），或依賴項改變時**
所執行的程式。

▶ 例如：抓第一次資料、開始計時器等。

### 2 清理函式（Cleanup，可選）

**當元件被清除後（從畫面上移除），在下一次新的 Setup 之前，**
所執行的程式。就是先清理上一輪留下的東西！

▶ 例如：清除計時器、取消監聽、中斷未完成的 API 請求等。

### 3 依賴陣列（Dependencies Array）

在陣列中設定「**依賴項**」，**也就是決定 useEffect 執行的觸發條件**
沒有設定的時候，只要元件觸發渲染更新，就會觸發這段程式
可以在陣列中限制要查看的值，當這些值改變，才重跑 Effect！

# useEffect

使用 useEffect 時，建議留意以下重點：
包含 useEffect 的執行時間點機、依賴陣列！

## 1 執行的時間點（非同步、不阻礙畫面渲染）

- useEffect 會在 React 完成 DOM 渲染、瀏覽器繪製出畫面之後才非同步執行。
- 不會阻礙瀏覽器渲染畫面，確保使用者體驗流暢。

## 2 依賴陣列的設定

- React 會比對「前一次渲染」與「本次渲染」傳入的依賴項是否改變

**不寫依賴陣列**

每次渲染完成後都會執行

監控全域變動（極少使用，**容易造成效能問題**）

```
useEffect(() => {})
```

**空陣列 [ ]**

僅在首次渲染進場後執行一次

首次載入打 API、設定全域監聽器

```
useEffect(() => {}, [])
```

**有依賴項 [a, b]**

首次渲染進場 + 當 a 或 b 改變時執行

搜尋關鍵字改變時重新打 API、分頁切換

```
useEffect(() => {}, [a, b])
```

03

3-4

# 列表渲染

## Rendering Lists

# .map()

在 React 中，若要渲染**一連串相同的元件清單**
可以利用 JavaScript 原生的 .map() 方法將資料轉換成 JSX 元素陣列，效果就像 Vue 的 v-for 一樣！

```jsx
// 元件
const Item = ({ num, name }) => {
  return (
    <li>
      <span>{num}.</span>
      <span>{name}</span>
    </li>
  );
};

// 手動一行一行列出來
export const ItemList = () => {
  return (
    <ul>
      <Item num={1} name="MantOu" />
      <Item num={2} name="Bun" />
      <Item num={3} name="Bread" />
    </ul>
  );
};
```

```jsx
// 資料陣列
const mockData = [
  { id: 1, name: "MantOu" },
  { id: 2, name: "Bun" },
  { id: 3, name: "Bread" },
];

// 透過 map 將陣列跑迴圈！
export const ItemList = () => {
  return (
    <ul>
      {mockData.map((item) => (
        <Item
          key={item.id}
          num={item.id}
          name={item.name}
        />
      ))}
    </ul>
  );
};
```

**1 準備資料陣列，呼叫 .map() 方法**

**2 設定輸入參數**

在 .map() 的箭頭函式中設定一個參數（例如 item），用來代表**當前輪到的那一筆資料**。

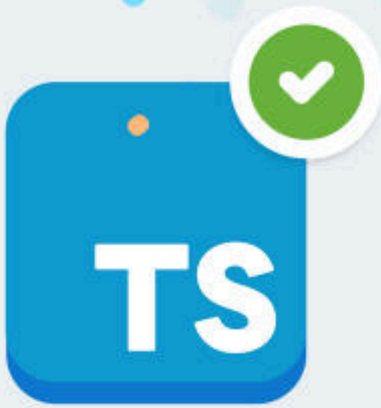
**3 用箭頭函式回傳要渲染的內容**

裡面放入要重複生成的元件結構（JSX）
**輸入參數**就可以直接給內部元件使用
並記得加上**不重複的 key** 作為 React 辨識標記！

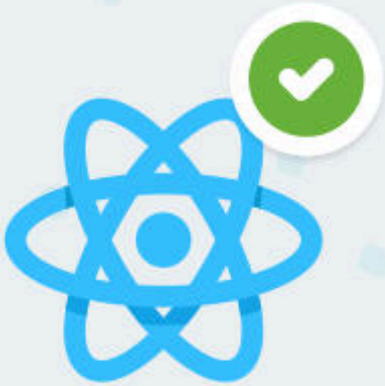
`.map((item, index) => ...)`
如果只是要簡單的數字序列，也可以利用 index
但千萬不要拿 index 當 key！




JavaScript ES6+

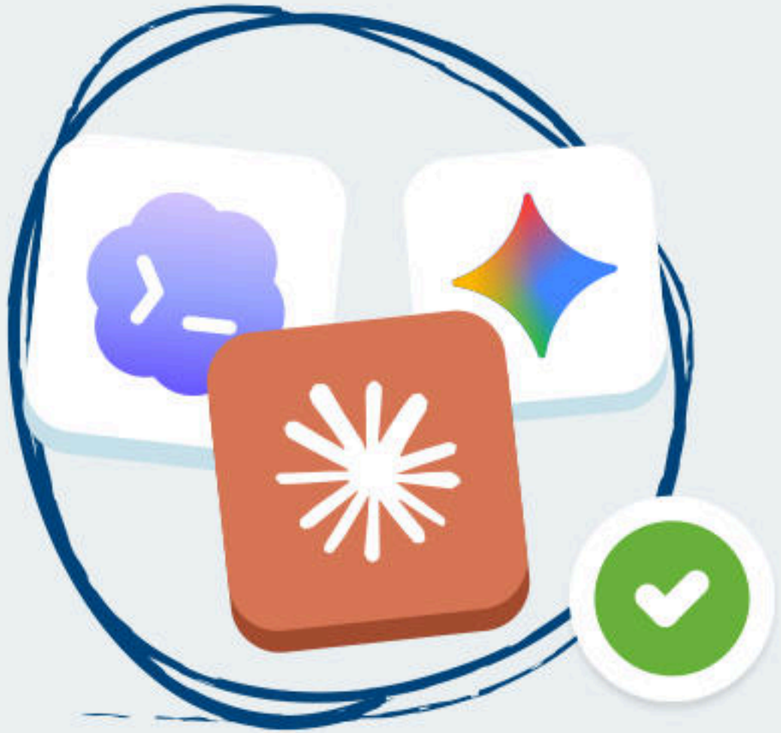



TypeScript



React


JS
TS



開發者 Buffet

# 一起來快樂惡補
# 基礎 JS、TS、React 吧

饅頭 / Mant0u

~~本次分享沒有介紹任何 AI 的部分~~

感謝大家





# GACHAGO!

# ガチャゴ!

點我